Introduction to

Python

Unit 1 — Weeks 1 & 2 | Control Statements · Lists · Tuples · Ranges · Dictionaries · Sets · I/O

2nd Year B.Tech CSE | Amrita Vishwa Vidyapeetham, Chennai

Reference: Allen B. Downey, Think Python, 2nd Ed., O'Reilly, 2016

Course Roadmap — Weeks 1 & 2

1 Intro to Python

Why Python? Setup, first program

2 Control Statements

if/elif/else, for, while, break, continue

3 Lists & Ranges

Creating, indexing, slicing, methods

4 Tuples

Immutable sequences, packing, unpacking

5 Dictionaries & Sets

Key-value pairs, set operations

6 Input & Output

input(), print(), format(), file I/O

1

Introduction to Python

Why Python? · Features · First Program

- High-level language
- Interpreted & Dynamic
- Simple readable syntax
- Huge standard library
- Cross-platform
- Open source & free

Why Python? — Features at a Glance

Simple Syntax

Reads almost like plain English. No curly braces, semicolons optional.

Interpreted

No compilation step — run code instantly; great for learning & prototyping.

Rich Libraries

NumPy, Pandas, Matplotlib, Django, Flask, scikit-learn and thousands more.

Cross-Platform

Same code runs on Windows, macOS, Linux without changes.

Data Science Ready

First choice for ML, AI, Data Analysis, and Scientific Computing.

Industry Standard

Used by Google, Netflix, Instagram, NASA, and most startups.

Your First Python Program

Key Concepts

`print()`	Built-in function to display output on screen
`# comment`	Anything after # is ignored by Python
`Strings`	Text enclosed in single '' or double "" quotes
`Variables`	Named containers — no type declaration needed
`Indentation`	Python uses spaces/tabs instead of { } braces

hello.py

```
# My first Python program
print("Hello, World!")

name = "Arun"
age  = 20
print("Name:", name)
print("Age :", age)
print(f"In 5 years, {name} will be
{age+5}")
```

OUTPUT

```
Hello, World!
Name: Arun
Age : 20
In 5 years, Arun will be 25
```

2

Control Statements

if · elif · else · for · while · break · continue · pass

- if / elif / else
- for loop
- while loop
- range()
- break & continue
- pass & nested

Conditional Statements — if / elif / else

Syntax & Rules

- Condition must evaluate to True or False
- Colon : is mandatory after condition
- Body must be indented (4 spaces recommended)
- elif = 'else if' — multiple conditions
- else block runs if ALL conditions are False
- Comparison: ==, !=, <, >, <=, >=
- Logical: and, or, not

grade_checker.py

```
marks = 78

if marks >= 90:
    print("Grade: O")
elif marks >= 80:
    print("Grade: A+")
elif marks >= 70:
    print("Grade: A")
elif marks >= 60:
    print("Grade: B+")
else:
    print("Grade: B")
```

OUTPUT

```
Grade: A
```

for Loop — Iterating Over Sequences

How it works

```
for variable in sequence:
    body
```

- Iterates over any sequence: string, list, tuple, range
- Loop variable takes each value one by one
- No need to track index manually
- range(n) → 0, 1, 2, … n-1
- range(a, b) → a, a+1, … b-1
- range(a, b, step) → with custom step
- enumerate() gives (index, value) pairs

for_examples.py

```
# Loop over a list
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)

# Loop with range
for i in range(1, 6):
    print(i, "x 2 =", i * 2)
```

OUTPUT

```
apple
banana
mango
1 x 2 = 2
2 x 2 = 4
3 x 2 = 6
4 x 2 = 8
```

💡 enumerate(fruits, 1) → (1,'apple'), (2,'banana'), (3,'mango')

while Loop · break · continue · pass

while_loop.py

```
# while loop
count = 1
while count <= 5:
    print("Count:", count)
    count += 1
print("Done!")
```

OUTPUT

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Done!
```

break_continue.py

```
# break — exit loop early
for n in range(1, 10):
    if n == 5:
        break
    print(n, end=" ")

# continue — skip current iteration
for n in range(1, 8):
    if n % 2 == 0:
        continue
    print(n, end=" ")
```

OUTPUT

```
1 2 3 4
1 3 5 7
```

break → exits the loop entirely | continue → skips current iteration, goes to next | pass → placeholder (does nothing)

Nested Conditions & Loop — Practical Example

number_classifier.py

```
# Classify numbers 1-15
for num in range(1, 16):
    if num % 15 == 0:
        print(num, "→ FizzBuzz")
    elif num % 3 == 0:
        print(num, "→ Fizz")
    elif num % 5 == 0:
        print(num, "→ Buzz")
    else:
        print(num, "→", num)
```

OUTPUT

```
1→1  2→2  3→Fizz  4→4  5→Buzz  6→Fizz  7→7
8→8  9→Fizz  10→Buzz  12→Fizz  15→FizzBuzz
```

multiplication_table.py

```
# Nested loop: multiplication table
for i in range(1, 4):
    for j in range(1, 4):
        print(i, "x", j, "=",
              i*j, end="\t")
    print()    # new line after each row
```

OUTPUT

```
1 x 1 = 1    1 x 2 = 2    1 x 3 = 3
2 x 1 = 2    2 x 2 = 4    2 x 3 = 6
3 x 1 = 3    3 x 2 = 6    3 x 3 = 9
```

3

Lists, Ranges & Tuples

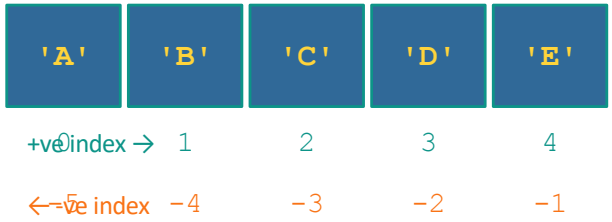
Ordered sequences — mutable vs immutable

- Creating lists
- Indexing & Slicing
- List methods
- range()
- Tuples
- Packing & Unpacking

Python Lists — Creating & Indexing

What is a List?

An ordered, mutable collection that can hold items of different types.



Slicing Cheat-Sheet

`lst[0]`	→ First element
`lst[-1]`	→ Last element
`lst[1:4]`	→ Index 1,2,3 (end exclusive)
`lst[:3]`	→ First 3 elements
`lst[2:]`	→ From index 2 to end
`lst[::2]`	→ Every 2nd element
`lst[::-1]`	→ Reverse the list

Python Lists — Creating & Indexing

list_basics.py

```
marks = [85, 92, 78, 95, 60]

print(marks[0])     # First
print(marks[-1])    # Last
print(marks[1:4])   # Slice
print(marks[:3])    # First 3
print(marks[::-1]) # Reverse

mixed = [1, "hello", 3.14, True]
print(mixed)
```

OUTPUT

```
85
60
[92, 78, 95]
[85, 92, 78]
[60, 95, 78, 92, 85]
[1, 'hello', 3.14, True]
```

List Methods — Modifying Lists

Built-in List Methods

`append(x)`	Add x to end	`reverse()`	Reverse in-place
`insert(i, x)`	Insert x at index i	`index(x)`	Find index of x
`extend(lst)`	Add all items of lst	`count(x)`	Count occurrences of x
`remove(x)`	Remove first x	`len(lst)`	Number of elements
`pop(i)`	Remove & return item at i	`copy()`	Shallow copy
`sort()`	Sort in-place	`clear()`	Remove all elements

List Methods — Modifying Lists

Built-in List Methods

`append(x)`	Add x to end	`reverse()`	Reverse in-place
`insert(i, x)`	Insert x at index i	`index(x)`	Find index of x
`extend(lst)`	Add all items of lst	`count(x)`	Count occurrences of x
`remove(x)`	Remove first x	`len(lst)`	Number of elements
`pop(i)`	Remove & return item at i	`copy()`	Shallow copy
`sort()`	Sort in-place	`clear()`	Remove all elements

list_methods.py

```
students = ["Ravi", "Priya", "Arun"]

students.append("Divya")
print(students)

students.insert(1, "Meena")
print(students)

students.remove("Arun")
print(students)

students.sort()
print(students)
```

OUTPUT

```
['Ravi', 'Priya', 'Arun', 'Divya']
['Ravi', 'Meena', 'Priya', 'Arun', 'Divya']
['Ravi', 'Meena', 'Priya', 'Divya']
['Divya', 'Meena', 'Priya', 'Ravi']
Count: 4
```

List Comprehension — Pythonic Way to Build Lists

Syntax

```
[ expression  for  item  in
iterable  if  condition ]
```

Creates a new list in a single readable line — replaces multi-line for-loops.

without_comprehension.py

```
# Old way (3 lines)
squares = []
for x in range(1, 6):
    squares.append(x**2)
print(squares)
```

OUTPUT

```
[1, 4, 9, 16, 25]
```

comprehension.py

```
# Squares
squares = [x**2 for x in range(1, 6)]
print(squares)

# Even numbers only
evens = [x for x in range(1,11) if x%2==0]
print(evens)

# Upper-case names
names = ["ravi", "priya", "arun"]
upper = [n.upper() for n in names]
print(upper)
```

OUTPUT

```
[1, 4, 9, 16, 25]
[2, 4, 6, 8, 10]
['RAVI', 'PRIYA', 'ARUN']
```

range() — Generating Number Sequences

Three Forms of range()

`range(stop)`

0 to stop-1 | range(5) → 0,1,2,3,4

`range(start, stop)`

start to stop-1 | range(2,7) → 2,3,4,5,6

`range(start,stop,step)`

with step size | range(0,10,2) → 0,2,4,6,8

range() is lazy — it does not create the full list in memory.
Use list(range(...)) to see all values.

range_examples.py

```
# Basic range
for i in range(5):
    print(i, end=" ")
print()

# With start & stop
for i in range(1, 6):
    print(i, end=" ")
print()

# With step
for i in range(0, 20, 4):
```

OUTPUT

```
0 1 2 3 4
1 2 3 4 5
0 4 8 12 16
5 4 3 2 1
```

Python Tuples — Immutable Sequences

Tuple vs List

Feature	Tuple	List
Syntax	( )	[ ]
Mutable?	No ✗	Yes ✓
Faster?	Yes ✓	Slower
Use When	Fixed data	Data changes

Common Uses

- Coordinates: (lat, lng)
- RGB colours: (255, 0, 0)
- Database records
- Function returning multiple values
- Dictionary keys (lists can't be keys)

tuples.py

```
# Creating tuples
point = (3, 7)
colors = ("red", "green", "blue")
single = (42,)    # Note: trailing comma!

# Accessing
print(point[0], point[1])
print(colors[-1])

# Packing & Unpacking
x, y = point
print(f"x={x}, y={y}")
```

OUTPUT

```
3 7
blue
x=3, y=7
red green blue
```

4

Dictionaries & Sets

Key-value pairs · Unique collections · Set algebra

- Creating dicts
- CRUD operations
- Dict methods
- Creating sets
- Set operations
- Frozenset

Python Dictionaries — Key : Value Pairs

Key Facts

- Created with { key: value } or dict()
- Keys must be unique & immutable (str, int, tuple)
- Values can be any type including lists/dicts
- Ordered (Python 3.7+) by insertion
- Access value by key — not by position
- KeyError raised if key doesn't exist
- Use .get(key) to avoid KeyError

Analogy: Like a real dictionary — look up a word (key) to get its meaning (value).

dict_basics.py

```
# Create dictionary
student = {
    "name": "Priya",
    "roll" : 101,
    "marks": [85, 90, 78]
}

# Access
print(student["name"])
print(student.get("age", "N/A"))

# Add / Update
student["age"] = 20
```

OUTPUT

```
Priya
N/A
{'name':'Priya','marks':[85,90,78,95],'age':20
}
```

Dictionary Methods & Iteration

Common Methods

`.keys()`	All keys	`.pop(k)`	Remove & return value for key
`.values()`	All values	`.setdefault(k, v)`	Get if key exists, else set and return default
`.items()`	All (key, value) pairs	`.copy()`	Shallow copy
`.get(k,d)`	Value or default	`len(d)`	Number of keys
`.update(d)`	Merge another dict	`k in d`	Membership check

dict_methods.py

```
scores = {"Ravi": 85, "Priya": 92, "Arun": 78}

# Iterate keys
for name in scores.keys():
    print(name)

# Iterate key-value pairs
for name, mark in scores.items():
    print(f"{name}: {mark}")

# Dict comprehension
passed = {k:v for k,v in scores.items()
          if v >= 80}
```

OUTPUT

```
Ravi  Priya  Arun
Ravi: 85
Priya: 92
Arun: 78
{'Ravi': 85, 'Priya': 92}
```

Python Sets — Unique Unordered Collections

Set Properties

Unordered	No fixed position / index
No Duplicates	Auto removes repeated values
Mutable	Can add/remove elements
Hashable items only	No lists/dicts as elements
Empty set	set() — not {} (that's a dict!)

Set Operations

`A | B` Union

`A & B` Intersection

`A - B` Difference

`A ^ B` Symmetric Diff

sets.py

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

print("Union      :", A | B)
print("Intersect  :", A & B)
print("Diff A-B   :", A - B)
print("Sym Diff   :", A ^ B)

# Remove duplicates from list
marks = [85,90,85,78,90,100]
unique = set(marks)
```

OUTPUT

```
Union      : {1,2,3,4,5,6,7,8}
Intersect  : {4, 5}
Diff A-B   : {1, 2, 3}
Sym Diff   : {1,2,3,6,7,8}
Unique: {100, 78, 85, 90}
```

Set Methods & Practical Use Cases

Set Methods

`.add(x)`
Add single element

`.update(s)`
Add multiple elements

`.remove(x)`
Remove x (KeyError if missing)

`.discard(x)`
Remove x (no error if missing)

`.pop()`
Remove & return arbitrary element

`.clear()`
Empty the set

`.copy()`
Shallow copy

`x in s`
Membership test

`.issubset(s)`
Is subset of s?

`.issuperset(s)`
Is superset of s?

`.isdisjoint(s)`
No common elements?

`frozenset(s)`
Immutable set

set_methods.py

```
enrolled = {"Ravi", "Priya", "Arun"}

enrolled.add("Divya")
print(enrolled)

enrolled.discard("Priya")
print(enrolled)

# Practical: find common students
batch_A = {"Ravi", "Priya", "Arun"}
batch_B = {"Priya", "Arun", "Meena"}
```

OUTPUT

```
{'Ravi', 'Priya', 'Arun', 'Divya'}
{'Ravi', 'Arun', 'Divya'}
Common: {'Priya', 'Arun'}
```

5

Input & Output in Python

input() · print() · Formatting · File I/O

- input() function
- Type conversion
- print() options
- f-strings
- format()
- File I/O

print() Options & String Formatting

print() Parameters

`sep=`	Separator between values (default: space)
`end=`	What to print at end (default: \n)
`file=`	Output destination (default: screen)
`flush=`	Force flush the output buffer

3 Ways to Format Strings

% operator: `"Hi %s" % name`

str.format(): `"Hi {}".format(name)`

f-string: `f"Hi {name}"` ← preferred

print_formatting.py

```
# sep & end
print(1, 2, 3, sep="-")
print("Hello", end=" ")
print("World")

# f-strings (preferred)
name = "Ravi"
marks = 92.5
print(f"Student: {name}")
print(f"Marks  : {marks:.1f}")
print(f"Grade  : {'O' if marks>=90 else 'A'}")
```

OUTPUT

```
1-2-3
Hello World
Student: Ravi
Marks  : 92.5
Grade  : O
Priya          85
```

File Input & Output

File Modes

`'r'`	Read (default) — file must exist
`'w'`	Write — creates/overwrites file
`'a'`	Append — adds to end of file
`'x'`	Create — error if file exists
`'rb'` / `'wb'`	Binary read / write
`'r+'`	Read and write

Always use with open(...) as f:
This ensures the file is closed automatically even if an error occurs.

file_io.py

```
# Write to file
with open("students.txt", "w") as f:
    f.write("Ravi 85\n")
    f.write("Priya 92\n")
    f.write("Arun 78\n")

# Read entire file
with open("students.txt", "r") as f:
    content = f.read()
print(content)
```

OUTPUT

```
Ravi 85
Priya 92
Arun 78

Ravi 85
Priya 92
Arun 78
```

Weeks 1 & 2 — Summary

Control Statements	if/elif/else, for, while, break, continue, pass, nested loops
Dictionaries	key:value pairs, .keys()/.values()/.items(), dict comprehension
Lists	Ordered, mutable, indexed, sliced, comprehension, 12+ methods
Sets	Unique items, union/intersection/difference, frozenset
range()	range(stop), range(start,stop), range(start,stop,step)
I/O	input(), print(sep, end), f-strings, format(), file read/write
Tuples	Immutable, packing, unpacking, faster than lists, hashable

Reference: Allen B. Downey, Think Python, 2nd Ed. · Practice on Python IDLE / VS Code / Jupyter Notebook